

Fundamentele limbajelor de programare

Programare funcțională. Codări Church în Lambda-calcul

Traian Florin Șerbănuță și Andrei Sipoș

Facultatea de Matematică și Informatică, DL Info

Anul II, Semestrul II, 2024/2025

Obiectiv: Cum reprezentăm date doar cu funcții?

Codări Church

- Reprezentarea datelor prin funcții (booleeni, numere, perechi etc.)
- Nu folosim structuri native — doar funcții lambda.
- Exemplu: valoarea „adevărat” = o funcție care returnează primul argument.

Idee de bază

Codăm un tip (recursiv) de date ca o potențialitate de a calcula ceva pe baza elementelor acelui tip.

- pentru tipuri nerekursive, ca o analiză de caz (case)
- pentru tipuri recursive ca o agregare (fold)

Booleeni

În Haskell

```
data Bool = False | True
```

```
-- / Case analysis for the 'Bool' type.
```

```
-- `bool f t p` evaluates to `f` when `p` is 'False',
```

```
-- and evaluates to `t` when `p` is 'True'.
```

```
bool :: a -> a -> Bool -> a
```

Booleeni

În Haskell

```
data Bool = False | True
```

```
-- | Case analysis for the 'Bool' type.  
-- `bool f t p` evaluates to `f` when `p` is 'False',  
-- and evaluates to `t` when `p` is 'True'.  
bool :: a -> a -> Bool -> a
```

Codare Church

Tipul **Bool** ::= $\forall a. a \rightarrow a \rightarrow a$

false : **Bool** ::= $\lambda x y. x$

true : **Bool** ::= $\lambda x y. y$

bool : $a \rightarrow a \rightarrow \mathbf{Bool} \rightarrow a$::= $\lambda f t p. p f t$

Bool: Operații derivate

if : **Bool** $\rightarrow$ $a \rightarrow a \rightarrow a ::= \lambda b t f. \mathbf{bool} f t b$

and : **Bool** $\rightarrow$ **Bool** $\rightarrow$ **Bool** $::= \lambda p q. \mathbf{if} p q \mathbf{false}$

or : **Bool** $\rightarrow$ **Bool** $\rightarrow$ **Bool** $::= \lambda p q. \mathbf{if} p \mathbf{true} q$

not : **Bool** $\rightarrow$ **Bool** $::= \lambda p. \mathbf{if} p \mathbf{false} \mathbf{true}$

Tipul Opțiune (Maybe)

În Haskell

```
data Maybe a = Nothing | Just a
```

```
-- | Case analysis for the 'Maybe' type.
```

```
-- `maybe n j m` returns `n` if `m` is 'Nothing';
```

```
-- otherwise, it applies the function `j` to the value
```

```
-- inside the 'Just' and returns the result.
```

```
maybe :: b -> (a -> b) -> Maybe a -> b
```

Tipul Opțiune (Maybe)

În Haskell

```
data Maybe a = Nothing | Just a
```

```
-- | Case analysis for the 'Maybe' type.  
-- `maybe n j m` returns `n` if `m` is 'Nothing';  
-- otherwise, it applies the function `j` to the value  
-- inside the 'Just' and returns the result.
```

```
maybe :: b -> (a -> b) -> Maybe a -> b
```

Codare Church

Tipul **Maybe** $a ::= \forall b. b \rightarrow (a \rightarrow b) \rightarrow b$

nothing : **Maybe** $a ::= \lambda n j. n$

just : $a \rightarrow$ **Maybe** $a ::= \lambda x. \lambda n j. jx$

maybe : $a \rightarrow (a \rightarrow b) \rightarrow$ **Maybe** $a \rightarrow b ::= \lambda n j m. m n j$

Operații cu Maybe

isNothing $\lambda m.\text{maybe true } (\lambda x.\text{false}) m$

maybeMap $\lambda f m.\text{maybe nothing } f m$

fromMaybe $\lambda d m.\text{maybe } d (\lambda x.x) m$

Tipul pereche

În Haskell

```
-- | 'uncurry' converts a curried function  
-- to a function on pairs.  
uncurry :: (a -> b -> c) -> (a, b) -> c
```

Tipul pereche

În Haskell

```
-- | 'uncurry' converts a curried function  
-- to a function on pairs.  
uncurry :: (a -> b -> c) -> (a, b) -> c
```

Codare Church

Tipul **Pair** $a \rightarrow b \rightarrow c$::= $\forall c. (a \rightarrow b \rightarrow c) \rightarrow c$
pair : $a \rightarrow b \rightarrow \mathbf{Pair} \ a \ b$::= $\lambda x y. \lambda f. f \ x \ y$
uncurry : $(a \rightarrow b \rightarrow c) \rightarrow \mathbf{Pair} \ a \ b \rightarrow c$::= $\lambda f \ p. p \ f$

Operații cu perechi

fst : **Pair** $a\ b \rightarrow a ::= \lambda p.\text{uncurry}(\lambda x\ y.x)\ p$

snd : **Pair** $a\ b \rightarrow b ::= \lambda p.\text{uncurry}(\lambda x\ y.y)\ p$

swap : **Pair** $a\ b \rightarrow \text{Pair } b\ a ::= \lambda p.\text{uncurry}(\lambda x\ y.\text{pair } y\ x)\ p$

curry : $(\text{Pair } a\ b \rightarrow c) \rightarrow a \rightarrow b \rightarrow c ::= \lambda f\ x\ y.f(\text{pair } x\ y)$

Liste

```
data List a = Cons a (List a) | Nil
```

```
-- / Right-associative fold of a list.  
foldr :: (a -> b -> b) -> b -> List a -> b
```

Codări Church

Tipul **List** $a ::= \forall b.(a \rightarrow b \rightarrow b) \rightarrow b \rightarrow b$

nil : **List** $a ::= \lambda c n.n$

cons : $a \rightarrow \text{List } a \rightarrow \text{List } a ::= \lambda x l.\lambda c n.c x (\text{foldr } c n l)$

foldr : $(a \rightarrow b \rightarrow b) \rightarrow b \rightarrow \text{List } a \rightarrow b ::= \lambda c n l.l c n$

Operații pe liste

null : **List** $a \rightarrow$ **Bool** ::= $\lambda l.$ **foldr** ($\lambda x b.$ **false**) **true** /

append : **List** $a \rightarrow$ **List** $a \rightarrow$ **List** a ::= $\lambda l1 l2.$ **foldr** **cons** /2 /1

map : $(a \rightarrow b) \rightarrow$ **List** $a \rightarrow$ **List** b ::= $\lambda f l.$ **foldr** ($\lambda x l'.$ **cons** ($f x$) l') **nil** /

filter : $(a \rightarrow$ **Bool**) $\rightarrow$ **List** $a \rightarrow$ **List** a ::=
 $\lambda f l.$ **foldr** ($\lambda x l'.$ **if** ($f x$) (**cons** $x l'$) l') **nil** /

uncons : **List** $a \rightarrow$ **Maybe** (**Pair** a (**List** a)) ::=
foldr ($\lambda x m.$ **just** (**pair** x (**maybe** **nil** (**uncurry** **cons**) m))) **nothing**

Numere Naturale

```
data Nat = Zero | Succ Nat
```

```
-- | `iter z s n` applies function `s` `n` times on initial value `z`
```

```
iter :: b -> (b -> b) -> Nat -> b
```

```
iter z s = go
```

```
  where
```

```
    go Zero = z
```

```
    go (Succ n) = s (go n)
```

Codări Church

Tipul **Nat** $a ::= \forall b. b \rightarrow (b \rightarrow b) \rightarrow b$

zero : **Nat** $::= \lambda z s. z$

succ : **Nat** $\rightarrow$ **Nat** $::= \lambda n. \lambda z s. s (\text{iter } z s n)$

iter : $b \rightarrow (b \rightarrow b) \rightarrow \text{Nat} \rightarrow b ::= \lambda z s n. n z s$

Operații pe numere naturale

isZero : **Nat** → **Bool** ::= $\lambda n.\text{iter true } (\lambda x.\text{false}) n$

plus : **Nat** → **Nat** → **Nat** ::= $\lambda m n.\text{iter } m \text{ succ } n$

mul : **Nat** → **Nat** → **Nat** ::= $\lambda m n.\text{iter zero } (\text{plus } m) n$

exp : **Nat** → **Nat** → **Nat** ::= $\lambda m n.\text{iter } (\text{succ zero}) (\text{mul } m) n$

prec : **Nat** → **Maybe Nat** ::=
 $\text{iter nothing } (\lambda m.\text{just } (\text{maybe zero succ } m))$

prec0 : **Nat** → **Nat** ::= $\lambda n.\text{fromMaybe zero } (\text{prec } n)$

minus0 : **Nat** → **Nat** → **Nat** ::= $\lambda m n.\text{iter } m \text{ prec0 } n$

lte : **Nat** → **Nat** → **Bool** ::= $\lambda m n.\text{isZero } (\text{minus0 } m n)$

La fel pentru orice tip de date (recursiv)

- Tipurile de date recursive se pot descrie ca puncte fixe
 - **Bool** $::= 1 + 1$ unde 1 este tipul singleton și + suma directă
 - **Maybe** $a ::= 1 + a$
 - **Pair** $a\ b ::= a \times b$ unde $\times$ este produsul
 - **List** $a ::= \mu x. a \times x + 1$ unde μ este cel mai mic punct fix
 - **Nat** $::= \mu x. 1 + x$
 - În general, $T ::= \mu x. F\ x$
- Aceste puncte fixe pot fi exprimate în lambda calcul (polimorfic)
 - $T ::= \forall \alpha. (F\ \alpha \rightarrow \alpha) \rightarrow \alpha$

Referințe:

- Philip Wadler (1990) Recursive types for free!
- Bartosz Milewski (2013) Understanding F-Algebras
- Bartosz Milewski (2017) F-Algebras

Concluzii și aplicații

- Codările Church arată că funcțiile sunt suficiente pentru a modela (aproape) orice tip de date (recursiv).
- Bază pentru teoria tipurilor și calculul computabilității.
- Înțelegerea acestor concepte ajută în proiectarea limbajelor de programare.